

Software Engineering Practice

Break this into **two parts**:

1. **The Essence of Practice** (the classic 4 steps)
2. **General Principles by David Hooker**

1. The Essence of Software Engineering Practice

This is inspired by the famous **Polya's Four Steps for Problem Solving**, but applied to software engineering:

It shows the **core thinking** every good developer should follow.

1. Understand the Problem

- Before writing any code, fully **understand what needs to be solved**.
- **Ask Questions**: What does the user really want?
- **Gather Requirements**: Use interviews, surveys, or study existing systems.
- **Clarify Inputs & Outputs**: What goes in? What comes out?

Example:

In FitTrack : *How does the step counter detect real steps? How do we avoid counting handshakes or random motion?*

2. Plan the Solution

- Once you understand the problem, **design how you'll solve it**.
- Break the solution into **modules** or **steps**
- Choose the right algorithms and data structures.
- Draw flowcharts, pseudocode, UML diagrams.

Example:

Plan how sensor data will be processed:

- Use Android's StepDetector API.
- Filter noise.
- Save steps in SQLite.

3. Carry Out the Plan

- Now, **implement** what you planned.
- Write clean, well-commented code.
- Test each part as you build.
- Follow coding standards.

Example:

Write the StepCounterService class in Java, connect it to a foreground service, test with a phone in your pocket.

4. Examine the Result

- Finally, **check if the solution works and meets requirements**.
- Test for bugs & edge cases.
- Compare outputs with expected results.
- Get user feedback.
- Improve if needed.

Example:

Check: Is the step count accurate when walking? What happens if you shake your phone? Are steps stored correctly in history?

- ✓ **These 4 steps form the heart of all good software engineering practice:**
- ✓ **Understand → Plan → Do → Check & Improve**

2.General Principles by David Hooker

Who is David Hooker?

- David Hooker is known for defining **7 timeless principles** that help software engineers **write better code** and **build better software**.
- They're not about *what* tools you use but *how* you think when you **design, write, and maintain** software.

Why do they matter?

Because bad software happens when people:

- Over-complicate things.
- Forget the original goal.
- Write messy code no one can understand later.
- Make software that breaks if you add new features.

Hooker's principles help you **avoid these traps**.

1. Reason It All Out

Meaning: *Think before you code.*

Don't just write code blindly — understand **why** you're doing it.

Example:

Before adding a new module in FitTrack, ask:

- Does it fit the goal of the app?
- Will it affect other modules?
- Is there a better way to do it?

Why it matters:

This prevents sloppy work and wasted effort.

2. Keep It Simple

Meaning: *Simple code is better code.*

A simple design is easier to build, test, and fix.

Example:

If you can write a function in 20 clear lines, don't squeeze it into a confusing 5-line “smart trick”. Future you (and teammates) will thank you!

Why it matters:

Complex code = more bugs, harder to maintain.

3. Maintain the Vision

Meaning: *Stay true to your project's purpose.*

Don't let random features or styles ruin the overall plan.

Example:

FitTrack is a fitness app. Suddenly adding social media chat? Not part of the core vision — unless planned. Stick to the core idea: helping users stay fit.

Why it matters:

Keeps your project focused, clear, and user-friendly.

4. What You Produce, Others Will Consume

Meaning: *Your code is not just for the computer .it's for other humans, too.*

@2415 Hira Nawaz Student at Zamindar College, Gujrat

Example:

Write clean code, use clear names (stepCounter not sc1), add helpful comments.

Think: If your friend looks at your code tomorrow, will they understand it easily?

Why it matters:

Other developers (or future you!) can fix bugs and add features faster.

5. Be Open to the Future

Meaning: *Software always changes.*

So design it to handle changes **without breaking everything**.

Example:

In FitTrack, maybe later you want to add a new module: a calories tracker. If your code is **modular**, you can plug it in easily.

Why it matters:

Future updates become cheaper and easier.

6. Plan Ahead for Reuse

Meaning: *Don't repeat yourself.*

Write modules that can be reused in other projects or other parts of the same app.

Example:

Your login & signup module can be reused in other apps or the same module can be reused for both users and admins.

Why it matters:

Saves time, effort, money less chance for bugs.

7. Think!

Meaning: *Don't work on autopilot.*

Keep questioning: Is there a better way? Is my solution clear, efficient, and right for the problem?

Example:

After writing your step count logic — pause and ask:

- Can it count hand shakes by mistake?
- Is there an algorithm that filters out fake steps better?

Why it matters:

Thinking actively = better software and better developer.

Putting it All Together

#	Principle	Real-Life Meaning
1	Reason It All Out	Understand every step, don't guess
2	Keep It Simple	Simple = better, clearer, safer
3	Maintain the Vision	Stick to your project's main goal
4	What You Produce, Others Will Consume	Write for humans, not just computers
5	Be Open to the Future	Make your software easy to extend
6	Plan Ahead for Reuse	Write reusable code
7	Think!	Always question and improve

Summar of David Hooker's 7 General Principles

#	Principle	Meaning (Easy Words)	Example
1	Reason It All Out	Think before you build. Understand every decision.	Don't just copy-paste code; know why it works.
2	Keep It Simple	Simpler is better. Avoid unnecessary complexity.	A clear 20-line method is better than a tricky 5-line mess.
3	Maintain the Vision	Stick to agreed goals and style. Don't lose focus.	If your app is for fitness tracking, don't randomly add shopping features.
4	What You Produce, Others Will Consume	Code is for people too, not just machines.	Use clear variable names, comments, and documentation.
5	Be Open to the Future	Design so the software can evolve easily.	Write modular code — you can add new workout types without rewriting everything.
6	Plan Ahead for Reuse	Reuse parts where possible. Don't reinvent the wheel.	Use the same login module for multiple apps.
7	Think!	Always think critically about what you're doing.	Ask: Is there a better way? Is this the best algorithm?